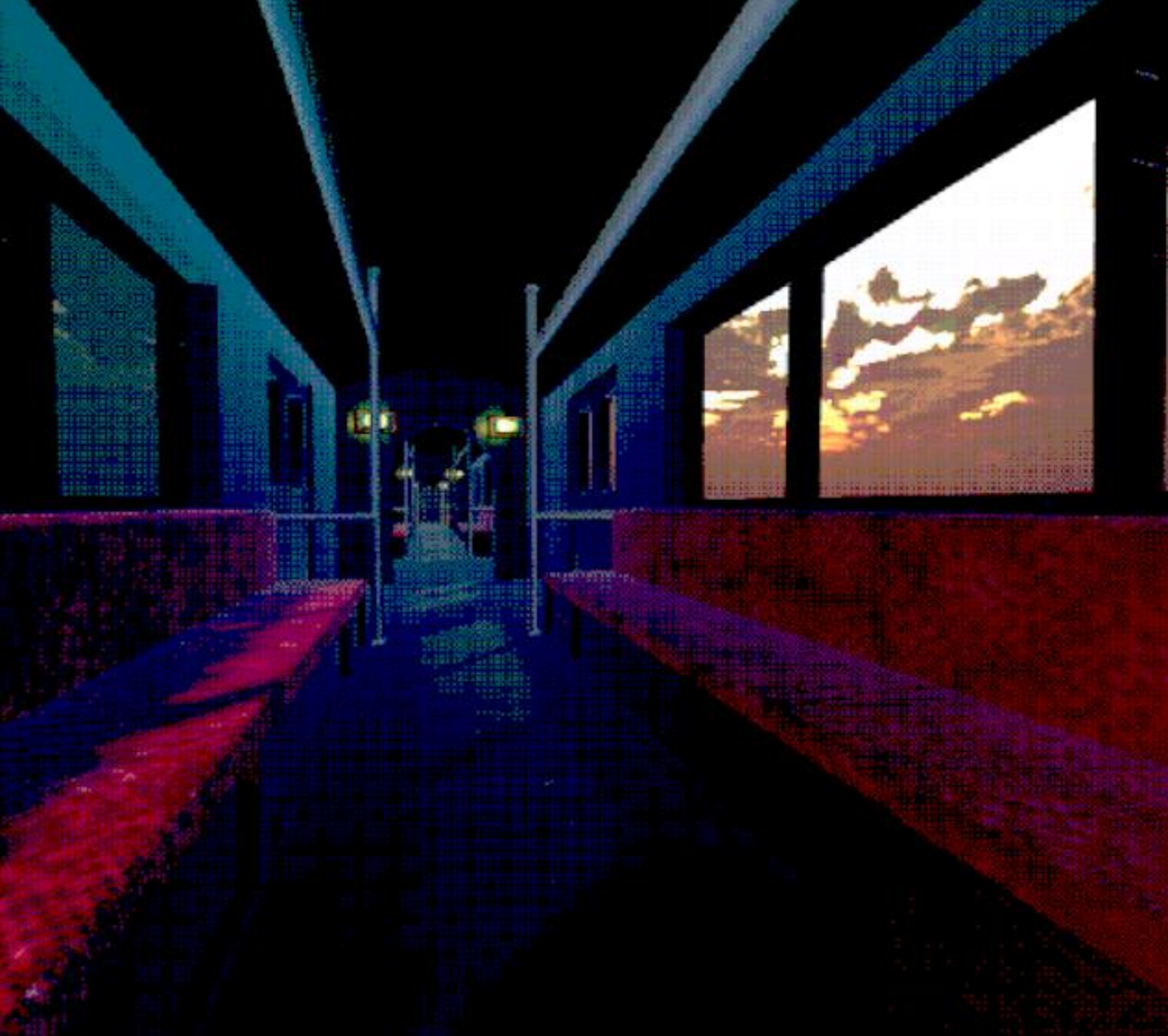


Manual técnico para Estaciones



Alumno: Silva Quintero Ricardo Santiago
Número de cuenta: 319012220
Fecha de entrega: 20/05/2026
Materia: Temas selectos de ingeniería en
computación 3 AR y VR 2026-2

Contenido

1. Descripción General del Sistema	3
2. Entorno de Desarrollo	3
3. Estructura del Proyecto	3
3.1 Organización de carpetas	3
3.2 Escenas del proyecto	4
4. Scripts — Descripción Técnica.....	4
PlayerController.cs	4
Inter.cs	5
InfiniteTrain.cs	5
StopTrain.cs	6
Teleport.cs	6
Portal.cs	7
LevelManager.cs	7
SaveSystem.cs	7
MirarJugador.cs	8
OjoMironGrande.cs.....	8
5. Sistema de Renderizado y Shader	8
5.1 Configuración del Universal Render Pipeline	8
5.2 Shader de filtro pixelado	9
6. Iluminación	10
6.1 Mapa1 — Vecindario Tranquilo: iluminación dinámica	10
6.2 Mapa2 — Baños: iluminación bakeada	11
7. Audio	12
8. Sistema de Partículas VFX	13
9. Generación de Builds	13
9.1 Build para PC — Windows.....	13
9.2 Build para Meta Quest 2 — APK	13
10. Repositorio y Control de Versiones	14
11. Consideraciones Técnicas y Limitaciones Conocidas	14

1. Descripción General del Sistema

Estaciones es un videojuego de exploración en 3D desarrollado en Unity 6000.3.7f1 con Universal Render Pipeline. El proyecto genera dos builds independientes desde el mismo código fuente: una aplicación ejecutable para PC en Windows y un paquete APK para Meta Quest 2 mediante OpenXR. Ambas builds comparten la totalidad de la lógica de juego, los assets y las escenas; la diferencia entre ellas reside en la configuración del renderer, los ajustes de calidad y el plugin de XR activo.

El juego está estructurado en torno a cinco escenas jugables que representan los espacios de la experiencia: el menú principal, la escena intermedia de transición, el Tren como hub central, y dos niveles de exploración denominados Mapa0 y Mapa1. Existe además una sexta escena, MapaPruebas, utilizada exclusivamente durante el desarrollo para probar mecánicas y assets de forma aislada sin afectar las escenas de producción.

2. Entorno de Desarrollo

El proyecto fue desarrollado íntegramente con las siguientes herramientas, versiones y configuraciones. Unity 6000.3.7f1 actúa como motor principal y entorno de integración. El pipeline de renderizado es Universal Render Pipeline en su versión incluida con Unity 6, configurado con dos Renderer Assets distintos: `pc_renderer` para la build de escritorio y `mobile_renderer` para la build de Meta Quest. Blender fue utilizado para el modelado, UV mapping, rigging y exportación de todos los assets 3D del proyecto en formato `.fbx`. FL Studio se empleó para la producción y exportación del audio ambiental original de cada escena. El control de versiones se gestiona con Git y GitHub, con Git LFS habilitado para archivos binarios pesados como modelos, texturas y audio.

Para la build de Meta Quest, el proyecto requiere adicionalmente el paquete XR Plugin Management con el proveedor OpenXR activo, el paquete Meta XR Core SDK y el Android Build Support instalado en el editor de Unity con el Android SDK y JDK correspondientes. La firma del APK se realiza mediante un keystore propio del equipo generado durante la configuración inicial del proyecto para plataforma Android.

3. Estructura del Proyecto

3.1 Organización de carpetas

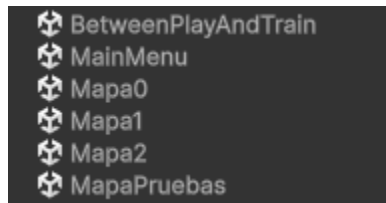
El proyecto sigue la estructura de carpetas estándar que genera Unity dentro del directorio Assets. Cada tipo de recurso tiene su propia carpeta dedicada para mantener el proyecto organizado y facilitar la navegación. La estructura principal es la siguiente.

Assets/	
├─ 3D/	Modelos importados desde Blender en formato <code>.fbx</code>
├─ Audios/	Clips de audio ambiental exportados desde FLStudio
├─ Escenas/	Las seis escenas del proyecto (<code>.unity</code>)

— Materials/	Materiales y configuraciones de shader
— Prefabs/	Objetos prefabricados reutilizables
— Scripts/	Todos los scripts en C#
— Settings/	Renderer Assets de URP y configuración de XR
— Textures/	Mapas de textura y lightmaps bakeados

3.2 Escenas del proyecto

El proyecto contiene seis escenas organizadas en la carpeta Escenas. MainMenu es la escena de inicio donde el jugador accede al menú principal e interactúa con el letrero de opciones. BetweenPlayAndTrain es la escena intermedia que muestra la ventana estilo Windows 7 durante aproximadamente 30 segundos antes de cargar el Tren. Mapa0 es el Tren, hub central del juego desde donde el jugador viaja entre mundos. Mapa1 corresponde al primer nivel jugable, el Vecindario Tranquilo. Mapa2 corresponde al segundo nivel jugable, los Baños. MapaPruebas es una escena auxiliar de desarrollo, no incluida en el flujo de juego de producción, utilizada para pruebas de assets y mecánicas de forma aislada.



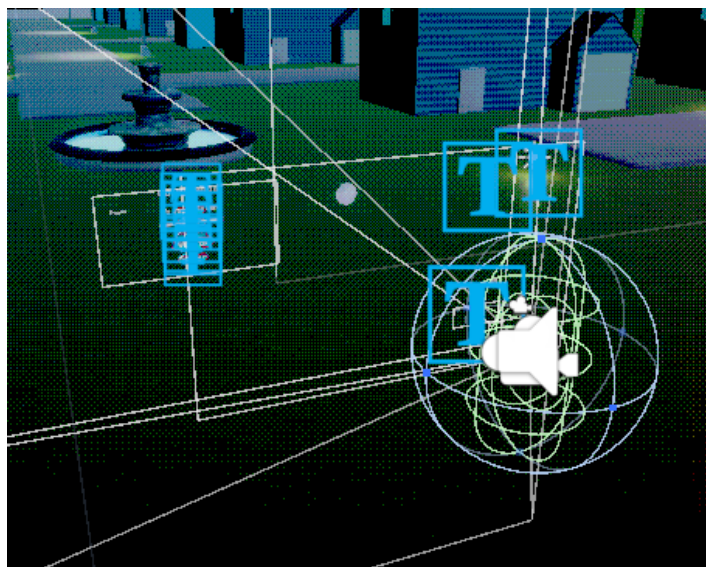
(Figura 1. Contenido de escenas)

4. Scripts — Descripción Técnica

Todos los scripts del proyecto están escritos en C# y se encuentran en la carpeta Assets/Scripts. A continuación, se describe la responsabilidad de cada uno, su ubicación dentro de la jerarquía de escenas y las dependencias relevantes entre ellos.

PlayerController.cs

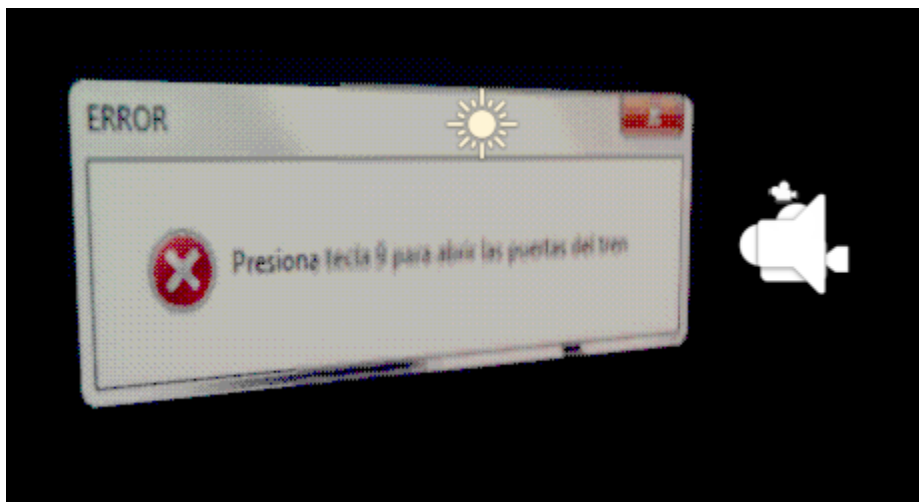
Es el script central del personaje jugable. Se encarga de leer las entradas del teclado y el ratón para trasladar y rotar al personaje en el espacio 3D. Gestiona el movimiento en las cuatro direcciones mediante las teclas WASD, la velocidad de carrera al mantener Shift izquierdo y la orientación de la cámara según el movimiento del ratón. Este script está presente en todas las escenas jugables y se adjunta al GameObject del jugador. No depende de otros scripts del proyecto, pero es referenciado por StopTrain y Teleport para controlar el estado del jugador durante las transiciones.



(Figura 2. Player dentro de la escena)

Inter.cs

Controla el comportamiento de la escena BetweenPlayAndTrain. Al iniciarse la escena, activa la ventana de estilo Windows 7 con el mensaje de instrucciones y lanza una corrutina con un temporizador de 30 segundos. Al cumplirse el tiempo, llama al SceneManager de Unity para cargar la escena Mapa0, que corresponde al Tren. Es un script de un solo propósito, sin dependencias externas.

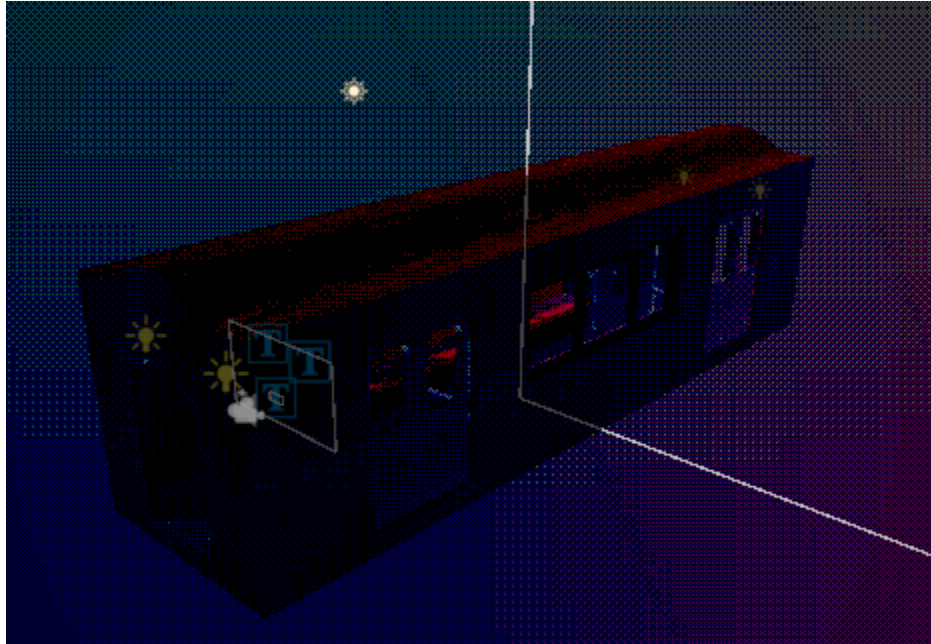


(Figura 3. Escena intermedia entre menú principal y El tren)

InfiniteTrain.cs

Implementa el efecto de tren infinito en la escena Mapa0. El script monitorea la posición del jugador a lo largo del eje de avance del tren y, conforme el jugador se desplaza,

instanciando nuevos vagones delante de él y destruye los que quedan demasiado atrás, manteniendo siempre una cantidad constante de vagones visibles en escena. Este enfoque evita tener una geometría de tren de longitud fija y mantiene el rendimiento estable independientemente del tiempo que el jugador permanezca en el Tren. El script referencia el prefab del vagón para su instanciación dinámica.



(Figura 4. Tren infinito en escena)

StopTrain.cs

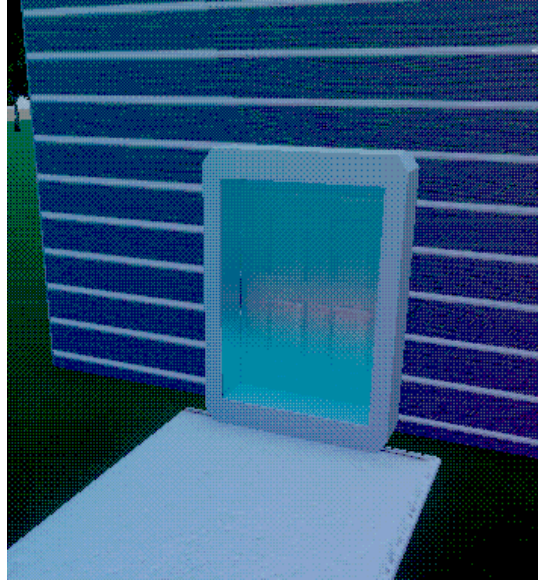
Detecta la pulsación de la tecla 9 y coordina la detención del tren. Al recibir el input, desactiva el componente InfiniteTrain para detener el movimiento y la instanciación de vagones, lanza la animación de frenado del tren, desactiva el AudioSource del motor y detiene la animación de la luz dinámica del interior del vagón. Una vez que las puertas están abiertas, habilita la zona de colisión que activa el script Teleport cuando el jugador la cruza. Centraliza la sincronización entre los sistemas de animación, audio, iluminación y lógica de progresión.

Teleport.cs

Gestiona la transición del jugador desde el Tren hacia el nivel correspondiente. Cuando el jugador cruza el volumen de colisión que StopTrain habilita al abrir las puertas, Teleport llama al SceneManager para cargar la escena del siguiente mundo. El script lleva un registro interno de qué nivel debe cargarse a continuación según el progreso del jugador. La transición es directa: no hay fundido a negro ni pantalla de carga visible; la nueva escena reemplaza a la anterior de forma inmediata.

Portal.cs

Es el equivalente inverso de Teleport: se encarga de regresar al jugador al Tren una vez que completa un nivel. El script está asociado a un objeto de portal que LevelManager activa cuando se alcanzan los diez fragmentos de estrella. Cuando el jugador entra en contacto con el portal, Portal.cs llama al SceneManager para cargar nuevamente la escena Mapa0. De esta forma, el ciclo de juego cierra: el jugador parte del Tren, completa el nivel y regresa al Tren para continuar al siguiente mundo.



(Figura 5. Portal en escena)

LevelManager.cs

Actúa como controlador de estado del nivel actual. Mantiene un contador de fragmentos de estrella recolectados y lo compara contra el objetivo de diez unidades. Cada vez que un fragmento es recogido, el objeto correspondiente notifica a LevelManager, que incrementa el contador y actualiza el elemento de interfaz X/10 visible en pantalla. Cuando el contador llega a diez, LevelManager activa el GameObject del portal, habilitando así el objeto gestionado por Portal.cs para que el jugador pueda regresar al Tren. Este script es el punto central de coordinación entre la recolección de objetos, la interfaz y la condición de completar el nivel.

SaveSystem.cs

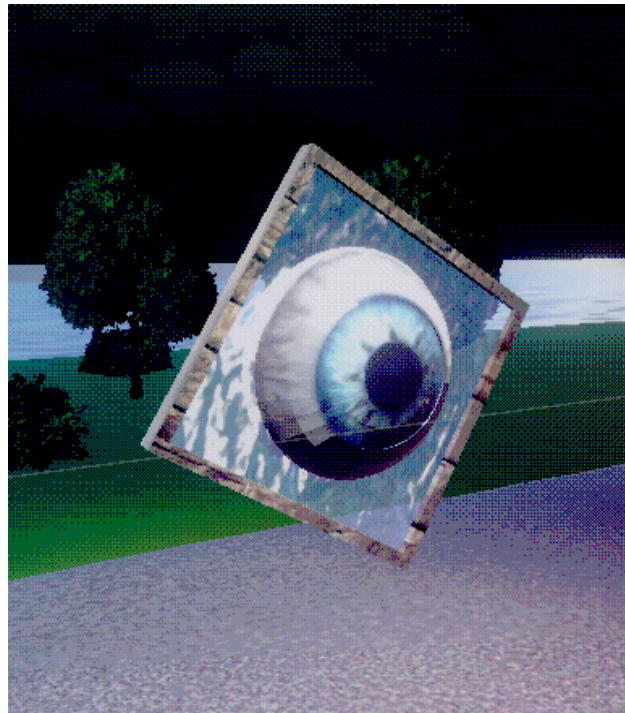
Implementa el sistema de guardado y carga de partida. Almacena el progreso del jugador de forma persistente utilizando los mecanismos de serialización de datos de Unity, registrando el nivel alcanzado y los fragmentos recolectados hasta el momento. Este sistema es el que alimenta la opción Cargar del menú principal, permitiendo al jugador retomar una partida anterior. El guardado se realiza de forma automática al completar cada nivel y regresar al Tren.

MirarJugador.cs

Script de comportamiento aplicado a las entidades NPC del juego. En cada frame, calcula la dirección entre el objeto donde está adjunto y la posición del jugador y rota el objeto para que siempre lo enfrente. El resultado es que las entidades dirigen su mirada hacia el jugador cuando este se encuentra en su radio de influencia, generando el efecto perturbador deseado sin necesidad de un sistema de IA complejo. Es un script ligero sin dependencias externas.

OjoMironGrande.cs

Es una variante extendida de MirarJugador.cs aplicada a entidades específicas que requieren un comportamiento de seguimiento más pronunciado o con parámetros distintos, como velocidad de rotación diferente o un rango de activación mayor. Comparte la misma lógica base de orientación hacia el jugador, pero con valores configurables independientes, lo que permite ajustar el comportamiento de cada tipo de entidad sin modificar el script compartido.



(Figura 6. Entidad con Script OjoMironGrande.cs)

5. Sistema de Renderizado y Shader

5.1 Configuración del Universal Render Pipeline

El proyecto utiliza dos `Renderer Assets` de URP almacenados en la carpeta `Assets/Settings`: `pc_renderer`, activo para la build de PC en Windows, y `mobile_renderer`, activo para la build de Meta Quest 2. Ambos comparten la misma configuración base de URP pero difieren en los ajustes de calidad, las sombras en tiempo real y las `Renderer Features` habilitadas según las capacidades de cada plataforma.

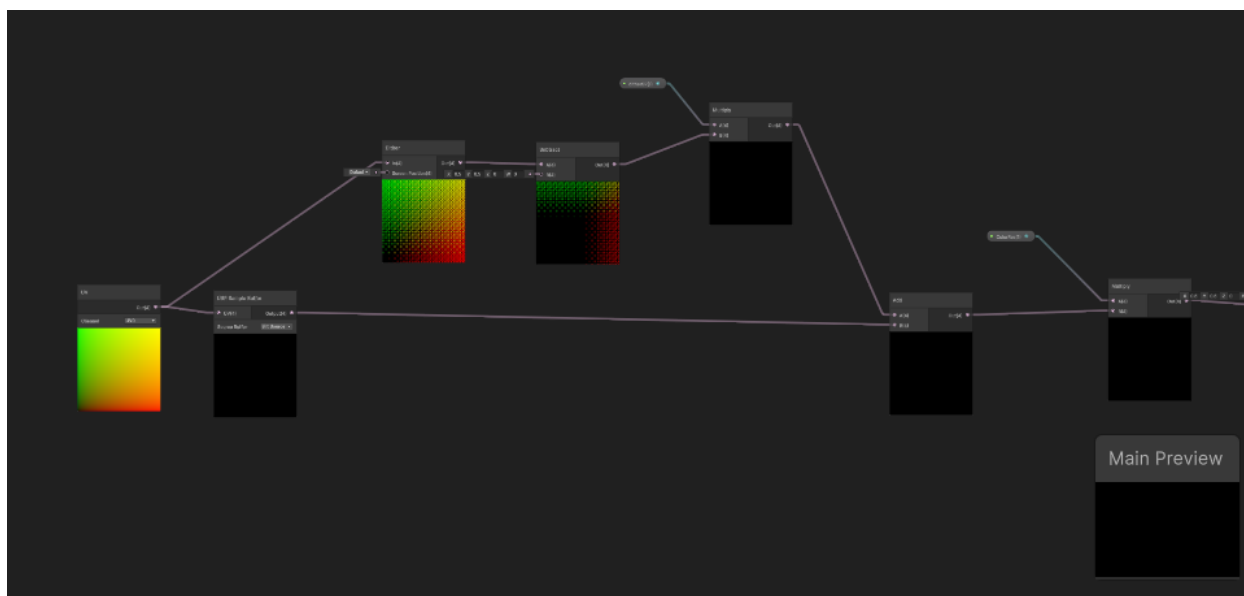
La selección del `renderer` activo se gestiona desde la `Quality Settings` del proyecto: el nivel de calidad asignado a la plataforma PC referencia `pc_renderer`, mientras que el nivel asignado a Android referencia `mobile_renderer`. Al generar cada build, Unity aplica automáticamente el `renderer` correspondiente sin requerir cambios manuales en el proyecto.

5.2 Shader de filtro pixelado

El `shader` de filtro pixelado es el elemento técnico más visible del proyecto y aplica sobre toda la imagen final que el jugador ve en pantalla. Fue creado con `Shader Graph` dentro del contexto de URP y está configurado como un `Full Screen Shader`, lo que significa que opera sobre la textura de salida de la cámara y no sobre geometría individual.

El funcionamiento técnico del `shader` es el siguiente. Toma como entrada la textura de la cámara renderizada, accesible en URP como `_CameraColorTexture`. Divide las coordenadas UV de la pantalla entre un valor de resolución de cuadrícula que determina el tamaño de cada bloque de píxeles. Aplica la función `Floor` sobre el resultado para redondear las coordenadas al entero inferior más cercano y las multiplica de nuevo por el valor de cuadrícula, generando bloques de color uniforme que simulan una resolución reducida. El parámetro de tamaño de bloque es configurable como propiedad del material, lo que permite ajustar la intensidad del efecto sin recompilar el `shader`.

El `shader` está integrado al pipeline mediante una `Full Screen Pass Renderer Feature` añadida manualmente tanto en `pc_renderer` como en `mobile_renderer`. Esta `Renderer Feature` inserta el pase del `shader` al final de la cadena de renderizado, después del postprocesado, de modo que el efecto pixelado se aplica sobre la imagen ya procesada incluyendo bloom, color grading y cualquier otro efecto de volumen activo. Esto garantiza que el filtro sea el último paso visual antes de presentar el frame al jugador.



6. Iluminación

El proyecto emplea una estrategia de iluminación mixta adaptada a las necesidades visuales y técnicas de cada espacio. La decisión de usar iluminación dinámica o bakeada en cada nivel no es arbitraria: responde directamente al tipo de geometría, la atmósfera deseada y las restricciones de rendimiento de la plataforma Meta Quest 2.

6.1 Mapa1 — Vecindario Tranquilo: iluminación dinámica

El Vecindario Tranquilo utiliza iluminación dinámica en tiempo real. Las fuentes de luz principales son los faroles del vecindario, configurados como Point Lights con sombras en tiempo real habilitadas. Esto permite que las sombras del personaje y de los elementos del entorno respondan al movimiento, reforzando la sensación de presencia del jugador dentro del espacio. El costo computacional de esta configuración es manejable en este nivel porque la geometría es low-poly y el número de luces dinámicas está acotado. Para la build de Meta Quest, las sombras en tiempo real de las luces puntuales se reducen en resolución desde la Quality Settings correspondiente a mobile_renderer para mantener el rendimiento dentro de los límites del hardware.

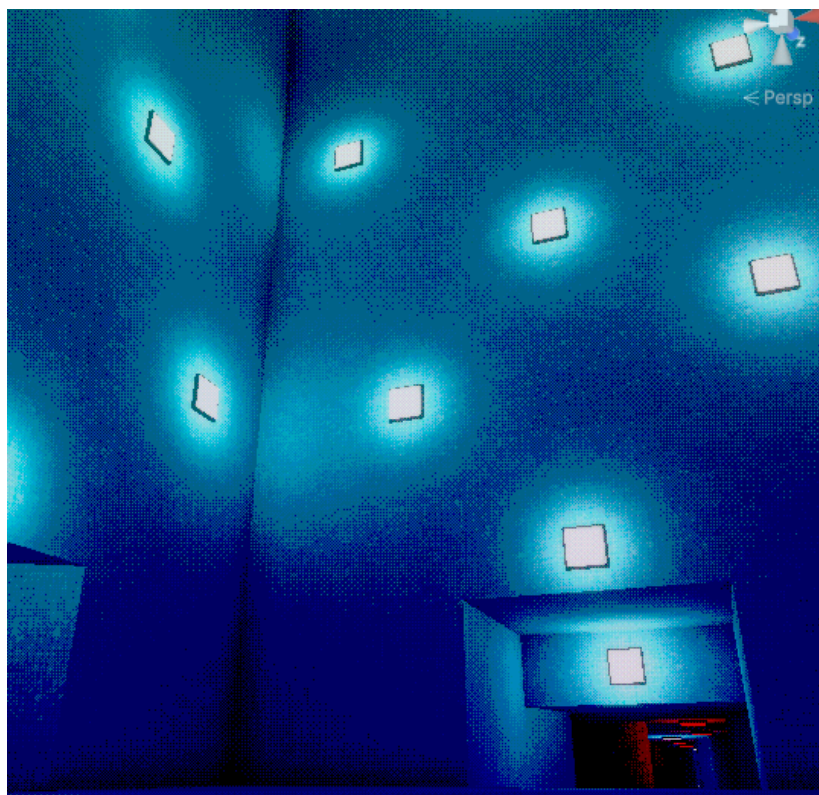


(Figura 8. Iluminación dinámica en Mapa1)

6.2 Mapa2 — Baños: iluminación bakeada

Los Baños utilizan iluminación completamente bakeada mediante el Lightmapper progresivo integrado en Unity. Todas las fuentes de luz del laberinto, luces fluorescentes de tipo Spot y Directional de baja intensidad, están marcadas como Mixed o Baked, y toda la geometría estática del nivel tiene el flag Contribute GI activado. El resultado del bake se almacena como lightmaps en la carpeta Assets/Textures y se aplica automáticamente al cargar la escena.

Esta decisión técnica tiene dos justificaciones. La primera es de atmósfera: la iluminación bakeada permite controlar con precisión los puntos exactos de luz y sombra en cada pasillo del laberinto, creando la oscuridad controlada y los focos tenues que definen la estética del nivel. La segunda es de rendimiento: al eliminar el cálculo de luces dinámicas en un entorno con geometría cerrada y muchos pasillos, se reduce drásticamente el número de draw calls y el costo de renderizado, lo que hace viable ejecutar este nivel en el Meta Quest 2 de forma fluida.



(Figura 8. Iluminación dinámica en Mapa1)

7. Audio

Todo el audio del proyecto fue producido originalmente en FL Studio y exportado en formato .mp3.

Cada escena tiene su propio AudioSource con el clip ambiental correspondiente configurado en modo loop. La escena del Tren tiene un segundo AudioSource dedicado al sonido del motor, que StopTrain.cs desactiva programáticamente al presionar la tecla 9. Esto garantiza que el silencio del motor forme parte de la respuesta coordinada de la mecánica de detención del tren junto con la animación y la luz.

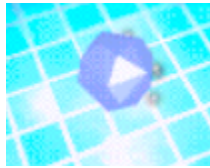
Los parámetros de volumen de música y efectos de sonido son ajustables desde el menú de pausa. Estos valores se gestionan a través de un Audio Mixer de Unity con dos grupos independientes: Music y SFX. Cada AudioSource del proyecto está enrutado al grupo correspondiente, de modo que los controles de volumen del menú afectan globalmente a todos los clips de su categoría sin necesidad de modificar cada fuente de audio individualmente.

8. Sistema de Partículas VFX

Los fragmentos de estrella fugaz utilizan un sistema de partículas de Unity (Particle System) adjunto como componente hijo del GameObject de cada fragmento. El sistema emite partículas luminosas de tamaño pequeño en un radio esférico alrededor del objeto, con un color cálido y una opacidad que varía a lo largo del ciclo de vida de cada partícula mediante una curva de alpha, generando el efecto de brillo pulsante que hace a los fragmentos identificables a distancia.

El sistema de partículas está configurado con emisión continua en loop mientras el objeto está activo en escena. Cuando el jugador recoge el fragmento, el script del objeto llama a LevelManager para registrar la recolección y posteriormente destruye el GameObject completo, incluyendo el Particle System hijo. Esto garantiza que las partículas desaparezcan de forma inmediata junto con el objeto sin dejar rastros visuales en escena.

El número de partículas por sistema está acotado deliberadamente para minimizar el impacto en el rendimiento cuando múltiples fragmentos están visibles simultáneamente en escena. En la build de Meta Quest, el módulo de Renderer del Particle System utiliza el shader de partículas optimizado de URP para móvil, que evita cálculos de iluminación adicionales sobre cada partícula.



(Figura 8. Sistema de partículas de fragmentos de estrella fugaz)

9. Generación de Builds

9.1 Build para PC — Windows

La build de PC se genera desde el Build Settings de Unity seleccionando la plataforma Windows, x86_64. El Renderer Asset activo para esta build es pc_renderer, que habilita sombras en tiempo real, el Full Screen Pass del shader pixelado y los efectos de postprocesado completos por Volume Stack. El resultado es una carpeta con el archivo ejecutable .exe y los datos de la aplicación. El peso total de la build es de aproximadamente 300 MB.

Para ejecutar el juego en PC basta con correr el archivo .exe directamente. No requiere instalación, registro ni conexión a internet. Los requisitos mínimos de hardware se detallan en el Manual de Usuario.

9.2 Build para Meta Quest 2 — APK

La build de Meta Quest se genera cambiando la plataforma activa a Android en el Build Settings y activando el proveedor OpenXR en el XR Plugin Management. El Renderer Asset activo para esta build es mobile_renderer, configurado con Single Pass Instanced

Rendering para reducir el costo del renderizado estéreo, Fixed Foveated Rendering en nivel medio para aliviar la carga de shaders en la periferia del campo visual, y resolución de sombras reducida respecto a la build de PC. El Full Screen Pass del shader pixelado también está habilitado en `mobile_renderer`.

El APK resultante pesa aproximadamente 300 MB y debe instalarse en el dispositivo mediante ADB o el Meta Quest Developer Hub con el dispositivo en modo desarrollador activado. Una vez instalado, el juego aparece en la biblioteca de aplicaciones del Quest bajo la categoría de fuentes desconocidas y puede ejecutarse de forma standalone sin conexión a PC.

10. Repositorio y Control de Versiones

A continuación, se presenta el enlace al repositorio de trabajo:

https://github.com/ULTRALEGENDB/Proyecto_Final_ARVR

11. Consideraciones Técnicas y Limitaciones Conocidas

El cambio de escena ocurre de forma directa mediante `SceneManager.LoadScene`, lo que puede producir un fotograma de pantalla negra perceptible durante la carga dependiendo del hardware del equipo. Esta decisión es intencional dentro del diseño de la experiencia.

El sistema `InfiniteTrain` instancia y destruye vagones en tiempo real. En hardware muy limitado, los picos de instanciación pueden causar micro-caídas de fotogramas. Para mitigarlo, el script mantiene un pool pequeño de vagones activos y evita instanciar más de uno por frame.

La escena `MapaPruebas` no debe incluirse en el `Build Settings` al generar las builds de producción. Si se incluye accidentalmente, aumenta el peso de la build sin aportar contenido jugable y puede exponer geometría incompleta.

En la build de Meta Quest, si el dispositivo no tiene el modo desarrollador activado, la instalación del APK mediante ADB fallará. Este paso debe completarse desde la aplicación Meta Quest en el teléfono vinculado al dispositivo antes del proceso de instalación.